A Task Allocation Method for Chiplet-Based CNN Accelerators

2025 IEEE 14th International Conference on Communications, Circuits and Systems (ICCCAS) | 979-8-3315-4477-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCCAS65806.2025.11102507

Congfeng Lv, Fen Ge*, Fang Zhou
College of Integrated Circuits
Nanjing University of Aeronautics and Astronautics
Nanjing, China
lvcongfeng@nuaa.edu.cn, gefen@nuaa.edu.cn, zfnuaa@ nuaa.edu.cn

***Abstract*—Chiplet-based Convolutional Neural Network (CNN) accelerators have emerged as a promising solution for efficient computational acceleration. However, communication between chiplets incurs higher costs compared to intra-chiplet communication. To address this challenge, this paper proposes a task allocation method designed to minimize inter-chiplet communication overhead. This method leverages the advantages of Time Division Multiplexing (TDM) and Space Division Multiplexing (SDM), consisting of two stages: task grouping and task partitioning. The CNN is divided into multiple task groups, which are allocated to different chiplets used SDM strategy. Subsequently, within each task group, *the* partitioning strategy of CNN layers is determined, and TDM strategy is used to allocate each CNN layer to processing units（PEs） within the chiplets. Experimental results demonstrate that the proposed method reduces inter-chiplet communication overhead by 59% compared to uniform task allocation method.**

Keywords—Chiplet; CNN; Task Allocation Method

I. INTRODUCTION

Convolutional Neural Networks (CNNs) are widely used in fields such as image processing, speech recognition and natural language processing[1][2]. However, as the depth of the network increases, the computational workload and memory access requirements of CNN models grow exponentially. Traditional System-on-Chip (SoC) designs have improved performance by increasing integration and expanding chip area. However, with the slowdown of Moore's Law, SoC faces limitations imposed by the area wall [3][4]. The increasing computational demands have led to the emergence of chiplet as a new paradigm for CNN accelerators.

Task allocation and mapping are the key steps in the design of chiplet-based CNN accelerators. Task allocation determines which computational tasks are executed by which chiplet, and an efficient task allocation method is crucial for reducing inter-chiplet communication. Studies have shown that the energy of data transmission between chiplets accounts for 30% of the total system energy consumption[5], making communication overhead become the primary bottleneck in the performance of chiplet-based accelerators. To address this issue, many studies have focused on optimizing task allocation method and mapping processes to reduce communication overhead. Existing task allocation and mapping strategies for CNN accelerators can be classified into two main categories: Time Division Multiplexing (TDM) and Space Division Multiplexing (SDM). The TDM strategy involves utilizing all computational resources of the chiplet accelerators to sequentially compute each layer, also known as layer sequential mapping. NN-Baton [6] proposes a task allocation and mapping strategy for chiplet-based CNN accelerators. It determines the partitioning strategy of each layer in the output feature map and sequentially maps each partitioned task to different chiplets and processing elements (PEs) within the chiplet to reduce unnecessary inter-chiplet communication volume. INDM [7] adopts layer sequential mapping and uses dynamic programming to optimize the partitioning strategy of each layer, reducing communication between chiplets during layer switches. The TDM strategy provides sufficient computational resources for each task, which reduces computation delay and improves performance. However, as chip scale expands, allocating a layer across too many computational resources results in lower resource utilization and increases data communication overhead between PEs. The SDM strategy allocates computational resources of the chiplet accelerator to multiple CNN layers simultaneously, enabling multiple tasks to be executed in parallel and pipelined way, also known as layer pipelining mapping. Simba[8] proposes a chiplet-based inference accelerator that simultaneously allocates multiple CNN layers to multiple chiplets and employs an unequal weight allocating method, where the computational load assigned to each chiplet is inversely proportional to the communication distance between the data storage areas. Gemini [9] employs layer pipelining mapping that partitions the output feature map of the CNN and maps each partition to multiple cores within the chiplet. These chiplet-based accelerators use SDM strategy, which significantly improves resource utilization. However, this can lead to mismatches between chiplet resources and computational or memory requirements, resulting in higher communication overhead and memory access cost.

To fully exploit the advantages of TDM and SDM strategies, this paper proposes a task allocation method for chiplet-based CNN accelerators that considers task allocation both across chiplets and within chiplets. By using the SDM strategy to assign task groups to different chiplets and applying the TDM strategy within each task group to allocate CNN layers to PEs

This work was supported by the National Key R&D Program of China (No.2022YFD2000700).

Authorized licensed use limited to: National Central University. Downloaded on October 03,2026 at 06:02:59 UTC from IEEE Xplore. Restrictions apply.

一種基於 Chiplet 的 CNN 加速器之任務分配方法

Congfeng Lv, Fen Ge*, Fang Zhou
College of Integrated Circuits
Nanjing University of Aeronautics and Astronautics Nanjing, China
lvcongfeng@nuaa.edu.cn, gefen@nuaa.edu.cn, zfnuaa@ nuaa.edu.cn

Abstract*—基於 Chiplet 的 Convolutional Neural Network (CNN) accelerators 已成為實現高效率計算加速的一種具前景的解決方案。然而，chiplets 之間的通訊相較於 intra-chiplet 通訊會帶來更高成本。為應對這項挑戰，本文提出一種旨在最小化 inter-chiplet communication overhead 的 task allocation 方法。此方法利用 Time Division Multiplexing（TDM）與 Space Division Multiplexing（SDM）的優勢，包含兩個階段：task grouping 與 task partitioning。CNN 被劃分為多個 task groups，並使用 SDM 策略分配至不同的 chiplets。接著，在每個 task group 內，決定 CNN layers 的 ***the partitioning 策略，並使用 TDM 策略將每個 CNN layer 分配至 chiplets 內的 processing units（PEs）。實驗結果證明，與 uniform task allocation 方法相比，所提出的方法可將 inter-chiplet communication overhead 降低 59%。**

Keywords—Chiplet; CNN; Task Allocation Method

I. 緒論

Convolutional Neural Networks (CNNs) 廣泛應用於影像處理、語音辨識與自然語言處理等領域[1][2]。然而，隨著網路深度增加，CNN 模型的計算工作量與記憶體存取需求呈指數成長。傳統 System-on-Chip (SoC) 設計透過提高整合度與擴大晶片面積來提升效能。然而，隨著 Moore's Law 放緩，SoC 面臨 area wall 所帶來的限制[3][4]。日益增加的計算需求促使 chiplet 成為 CNN 加速器的新典範。

Task allocation 與 mapping 是設計 chiplet-based CNN accelerators 的關鍵步驟。Task allocation 決定哪些計算任務由哪個 chiplet 執行，而高效的 task allocation 方法對於降低 inter-chiplet communication 至關重要。研究顯示，chiplets 之間資料傳輸的 energy 佔總系統 energy consumption 的 30%[5]，使得 communication overhead 成為 chiplet-based accelerators 效能的主要瓶頸。為了解決此問題，許多研究專注於優化 task allocation 方法與 mapping 流程，以降低 communication overhead。現有的 task 針對 CNN 加速器的 allocation 與 mapping 策略可分為兩大類：Time Division Multiplexing (TDM) 與 Space Division Multiplexing (SDM)。TDM 策略涉及利用 chiplet 加速器的所有計算資源來依序計算每一 layer，也稱為 layer sequential mapping。NN-Baton [6] 提出一種針對 chiplet-based CNN 加速器的 task allocation 與 mapping 策略。它決定 output feature map 中每一 layer 的 partitioning 策略，並將每個 partitioned task 依序映射到不同的 chiplets 以及 chiplet 內的 processing elements (PEs)，以減少不必要的 inter-chiplet 通訊量。INDM [7] 採用 layer sequential mapping，並使用 dynamic programming 來最佳化每一 layer 的 partitioning 策略，減少 layer 切換期間 chiplets 之間的通訊。TDM 策略為每個 task 提供充足的計算資源，可降低計算延遲並改善效能。然而，隨著 chip 規模擴大，將一個 layer 分配到過多計算資源會導致較低的資源利用率，並增加 PEs 之間的資料通訊開銷。SDM 策略將 chiplet 加速器的計算資源同時分配給多個 CNN layers，使多個 tasks 能以 parallel 與 pipelined 方式執行，也稱為 layer pipelining mapping。Simba[8] 提出一種 chiplet-based inference 加速器，可同時將多個 CNN layers 分配給多個 chiplets，並採用 unequal weight allocating 方法，其中分配給每個 chiplet 的計算負載與資料儲存區域之間的通訊距離成反比。Gemini [9] 採用 layer pipelining mapping，將 CNN 的 output feature map 分割，並將每個 partition 映射到 chiplet 內的多個 cores。這些 chiplet-based 加速器使用 SDM 策略，可顯著改善資源利用率。然而，這可能導致 chiplet 資源與計算或記憶體需求之間的不匹配，進而產生更高的通訊開銷與記憶體存取成本。

為了充分利用 TDM 和 SDM 策略的優勢，本文提出一種針對 chiplet-based CNN accelerators 的 task allocation 方法，其同時考量跨 chiplets 與 chiplet 內的 task allocation。透過使用 SDM 策略將 task groups 分配給不同 chiplets，並在每個 task group 內套用 TDM 策略，以將 CNN layers 分配給 PEs。

This work was supported by the National Key R&D Program of China (No.2022YFD2000700).

Authorized licensed use limited to: National Central University. Downloaded on October 03,2026 at 06:02:59 UTC from IEEE Xplore. Restrictions ap層。

within the chiplet, the proposed method minimizes both inter-chiplet and intra-chiplet communication overheads, thereby enhancing the performance of the accelerator.

II. Chiplet-based CNN Accelerator Architecture

Before discussing the task allocation method, a brief overview of the chiplet-based CNN accelerator architecture is necessary. A representative chiplet-based accelerator architecture is shown in Figure 1. This accelerator is composed of computing chiplets and I/O chiplets at the packaging level (the number of chiplets in Fig. 1 is for illustration, and the actual quantity can be adjusted). These chiplets are interconnected through Network on Interposer (NoI). I/O chiplets primarily handle communication with external DRAM. At the chiplet level, each computing chiplet contains multiple PEs, which are interconnected through Network on Chip (NoC) to facilitate internal connections and data transmission.

Based on this architecture, the task allocation problem researched in this paper aims to determine how the computational tasks of each layer of the CNN should be allocated to specific chiplets and corresponding PEs, with the goal of minimizing the communication overhead both within and between chiplets, particularly focusing on optimizing inter-chiplet communication overhead. To a certain extent, this task allocation method can also be applied to other architectures of chiplet-based CNN accelerators with output-stationary flow.

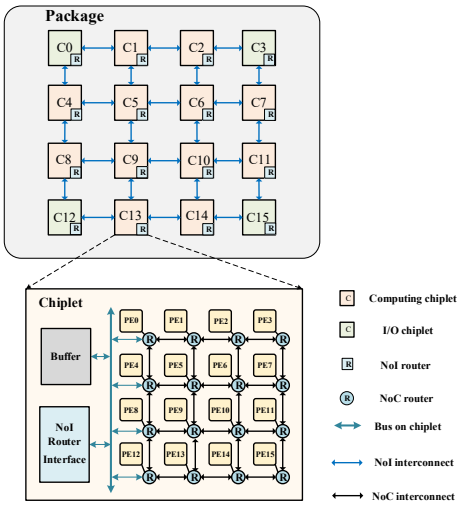


Fig. 1. Chiplet-based CNN accelerators architecture with mesh structure.

III. The Proposed Task Allocation Method

The proposed task allocation method for chiplet-based CNN accelerators can be divided into two main stages: task grouping and task partitioning, as shown in Fig. 2. In the task grouping stage, the key principle is to group tasks with complex data dependencies together, so that this group can be assigned to a chiplet. In the task partitioning stage, based on the computational requirements of each task group, the number of required PEs is precisely determined and further planning is done for the specific partitioning of tasks for each layer. The computational tasks for the output feature maps of each layer are partitioned, and the partitioning results are assigned to a PE to ensure efficient resource utilization and computational load balancing. Based on the task allocation results, the subsequent steps will determine which PEs will execute the assigned tasks, which is referred to as mapping. Mapping is a part of the follow-up research work and is not the focus of discussion here.

A. Task grouping

To clearly introduce the proposed task grouping method, we first provide the following definitions.

Definition 1: The application task graph DAG (V, E) is given, where each vertex $v_i \in V$ represents an operation layer (input layer, convolution layer, output layer, etc.) of a CNN application, and each edge $e_{ij} \in E$ represents the dependency relationship from v_i to v_j. The number of nodes is k, and each node v_i has a computational load o_i, representing the number of multiply-accumulate operations required for convolution, pooling, or activation. The weight of each edge w_{ij} represents the communication overhead between nodes v_i and v_j, which is characterized by the number of neuron connections between layers.

Definition 2: The set of CNN task groups is denoted as $G=\{G_1, G_2, \ldots, G_n\}$, where S_i represents a group in the DAG application task graph. There should be no overlap between different groups, meaning the intersection of any two groups must be empty, i.e., $G_i \cap G_j = \varnothing$, $\bigcup_{i=1}^{n} G_i = G$.

Definition 3: The chiplet-based CNN accelerator architecture graph CAG (C, L) is given, where each vertex $c_i \in C$ represents a chiplet, and each edge $l_i \in L$ represents the interconnection between chiplets c_i and c_j. Each node c_i contains p PEs, and each PE provides a maximum computational resource m (number of MACs). In this paper, a homogeneous design is adopted, meaning each chiplet has the same computational resources, which are uniformly represented by $C_{\max}$, $C_{\max} = p \times m$.

Based on the above definitions, the task grouping problem for chiplet-based CNN accelerators can be described as follows: Given the application task graph DAG and the chiplet-based CNN accelerator architecture graph CAG, the goal is to find an optimal task grouping set G.

The optimization objective for task grouping is to evenly balance the computational load across groups while minimizing the communication data between them. This ensures that the computational load is distributed uniformly across chiplets, preventing hotspots and reducing communication overhead. Hence, the objective function is defined as the cost, which is calculated by the following formula.

$$\cos t = Var(G) \cdot Cut(G) \tag{1}$$

$$Var(G) = \frac{1}{n} \sum_{i=1}^{n} \left(C_i - \overline{C}\right)^2 \tag{2}$$

$$C_i = \sum_{u \in G_i} o_u \tag{3}$$

$$\mathrm{Cut}(G) = \sum_{u \in G_i, v \in G_j, i \neq j} w_{uv} \tag{4}$$

Authorized licensed use limited to: National Central University. Downloaded on October 03,2026 at 06:02:59 UTC from IEEE Xplore. Restrictions apply.

在 chiplet 內，所提出的方法同時最小化 inter-chiplet 與 intra-chiplet communication overheads，從而提升 accelerator 的效能。

II. 基於 Chiplet 的 CNN 加速器架構

在討論任務分配方法之前，有必要先簡要概述基於 chiplet 的 CNN 加速器架構。圖 1 展示了一種具代表性的基於 chiplet 的加速器架構。此加速器在封裝層級由運算 chiplet 與 I/O chiplet 組成（圖 1 中的 chiplet 數量僅供說明，實際數量可調整）。這些 chiplet 透過 Network on Interposer (NoI) 互連。I/O chiplet 主要負責與外部 DRAM 的通訊。在 chiplet 層級，每個運算 chiplet 包含多個 PE，這些 PE 透過 Network on Chip (NoC) 互連，以促進內部連接與資料傳輸。

基於此架構，本論文所研究的 task allocation 問題旨在決定 CNN 每一層的 computational tasks 應如何分配到特定的 chiplets 及對應的 PEs，目標是最小化 chiplets 內部與 chiplets 之間的 communication overhead，尤其著重於最佳化 inter-chiplet communication overhead。在某種程度上，此 task allocation 方法亦可應用於其他具備 output-stationary flow 的 chiplet-based CNN accelerators 架構。

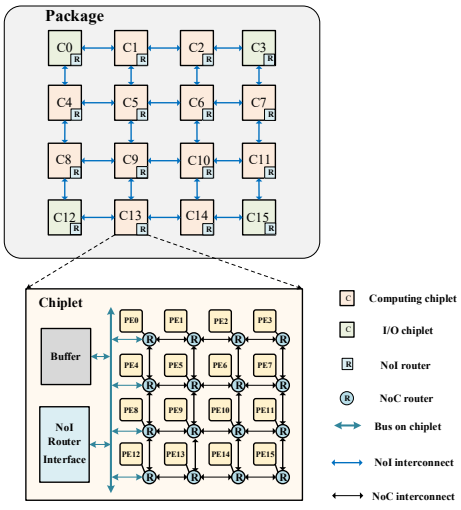


Fig. 1. 基於 Chiplet 的 mesh 結構 CNN 加速器架構。

III. 所提出的任務分配方法

所提出的針對基於 chiplet 的 CNN 加速器之 task allocation 方法可分為兩個主要階段：task grouping 與 task partitioning，如 Fig. 2 所示。在 task grouping 階段，關鍵原則是將具有複雜 data dependencies 的任務分組在一起，使此群組可被分配到一個 chiplet。在 task partitioning 階段，根據每個 task group 的計算需求，精確決定所需的 PEs 數量，並針對每一層任務的具體分割進一步規劃。每一層輸出 feature maps 的計算任務會被分割，且分割結果會被分配給一個 PE 以確保高效的 resource utilization 與 computational load balancing。根據 task allocation 的結果，後續步驟將決定哪些 PEs 會執行所指派的任務，這稱為 mapping。Mapping 是後續研究工作的一部分，並非此處討論的重點。

A. *Task grouping*

為了清楚地介紹所提出的 task grouping 方法，我們首先提供以下定義。

定義 1：給定應用任務圖 DAG（V, E），其中每個 vertex $v_i \in V$ 代表 CNN 應用的一個 operation layer（input layer、convolution layer、output layer 等），而每個 edge $e_{ij} \in E$ 代表從 v_i 到 v_j 的 dependency relationship。node 的數量為 k，且每個 node v_i 具有一個 computational load o_i，代表 convolution、pooling 或 activation 所需的 multiply-accumulate operation 數量。每個 edge w_{ij} 的 weight 代表 node v_i 與 v_j 之間的 communication overhead，其特徵為 layer 之間的 neuron connection 數量。

定義 2：CNN task groups 的集合表示為 $G=\{G_1, G_2, \ldots, G_n\}$，其中 S_i 代表 DAG application task graph 中的一個 group。不同的 group 之間不應有重疊，這意味著任意兩個 group 的交集

must be empty, i.e., $G_i \cap G_j = \varnothing$, $\bigcup_{i=1}^{n} G_i = G$.

定義 3：給定基於 chiplet 的 CNN accelerator 架構圖 CAG (C, L)，其中每個頂點 $c_i \in C$ 代表一個 chiplet，且每個邊 $l_i \in L$ 代表 chiplet c_i 與 c_j 之間的互連。每個節點 c_i 包含 p 個 PE，且每個 PE 提供最大計算資源 m（MACs 的數量）。本文採用同質設計，意即每個 chiplet 具有相同的計算資源，並統一以 $C_{\max}, _{\max} = p \times m$ 表示。

根據上述定義，針對基於 chiplet 的 CNN 加速器的 task grouping 問題可描述如下：給定應用程式 task graph DAG 以及基於 chiplet 的 CNN 加速器 architecture graph CAG，目標是找到一個最佳的 task grouping 集合 G。

task grouping 的 optimization objective 是在 groups 之間均勻平衡 computational load，同時最小化它們之間的 communication data。這可確保 computational load 均勻分佈於 chiplets，避免 hotspots 並降低 communication overhead。因此，objective function 定義為 cost，其由以下公式計算。

$$\cos t = Var(G) \cdot Cut(G) \tag{1}$$

$$Var(G) = \frac{1}{n} \sum_{i=1}^{n} \left(C_i - \overline{C}\right)^2 \tag{2}$$

$$C_i = \sum_{u \in G_i} o_u \tag{3}$$

$$\mathrm{Cut}(G) = \sum_{u \in G_i, v \in G_j, i \neq j} w_{uv} \tag{4}$$

Authorized licensed use limited to: National Central University. Downloaded on October 03,2026 at 06:02:59 UTC from IEEE Xplore. Restrictions apply.

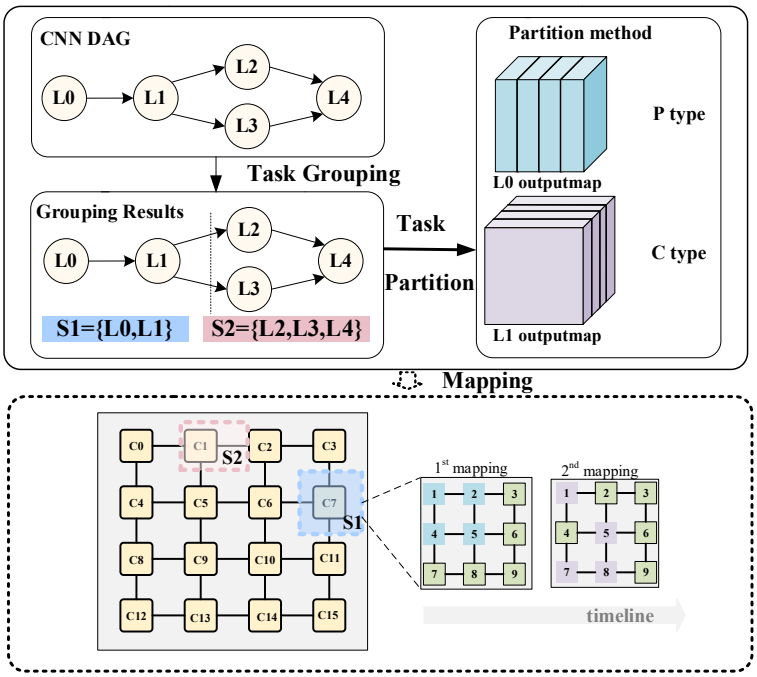


Fig. 2. Task allocation process for Chiplet-based CNN accelerators.

Var(*G*) represents the variance of the inter-group computational load, where C_i is the computational load of the *i-th* group, and $\overline{C}$ is the average computational load of all groups. *Cut*(*G*) represents the sum of the weights of the inter-group.

This paper proposes a task grouping method based on dynamic programming (*DP*), as shown in Algorithm 1. The core idea of this method is to maintain load balancing within each group while ensuring that the computational load of each task group does not exceed the preset maximum limit C_{max}, and the inter-group communication data volume is minimized. The specific steps are as follows: The method first clusters CNN application task graph (DAG) based on the distance between each node and the input node. This transforms the originally complex task graph with non-linear dependencies into a structure with relatively simple linear relationships, thus reducing the time complexity of the dynamic programming algorithm and simplifying the subsequent task grouping process.

Algorithm1 DAG Grouping with DP

Input: DAG, C_{max}
Output: $G=\{G_1,G_2,\ldots,G_n\}$
linearize DAG
temGroup[]={ }
for *i* = 1 **to** *k* **do**
  for *j*=1 **to** *i* **do**
    if $C(i,j) \le C_{max}$
    tempcost[*i*]= cost[*j*]+*cost(j,i)*
    if *tempcost[i]*<*cost[i]*
      cost[i]←*tempcost[i]*
      tempGroup[i]={tempGroup[j],[vj,…,vi] }
end for
G←*tempGroup[n]*
Return *S*

Next, DP is used to group the nodes in the task graph, partitioning nodes with consecutive indices into multiple subgraphs (i.e., task groups). The computational load of each subgraph must be controlled within the maximum limit C_{max} to ensure effective load balancing under hardware resource constraints. At the same time, the algorithm must minimize data transmission across groups to reduce the communication volume between chiplets.

B. Task partitioning

After completing the task grouping, we assign each task group to a chiplet. The next step is to determine the task partitioning within the chiplet, which involves allocating the computational tasks of each CNN layer within the task group to the PEs of the chiplet. Additionally, it is essential to plan how to partition the output feature map. Therefore, we propose a task partitioning method that first determines the number of PEs required for a group, and then defines the partitioning strategy for each layer within the group, specifying which part of the computation task of each layer is allocated to each PE.

1) Calculation of the required number of PEs

During task allocation, the goal is to make the computational load assigned to each PE, denoted as m_i, as close as possible to but not exceeding the limit value *m*. Let *H*, *W*, and *C* represent the height, width, and number of channels of the output feature map of a certain layer, respectively, and *F* denote the window size for convolution or pooling, while *S* represents the stride of convolution or pooling. By calculating, the total computational load of the group is denoted as C_i. The number of PEs (N) required for the group can be obtained using the following formula, where ε is the largest integer that meets the condition.

$$N = \varepsilon \tag{5}$$

$$m_i = C_i / \varepsilon \tag{6}$$

Authorized licensed use limited to: National Central University. Downloaded on October 03,2026 at 06:02:59 UTC from IEEE Xplore. Restrictions apply.

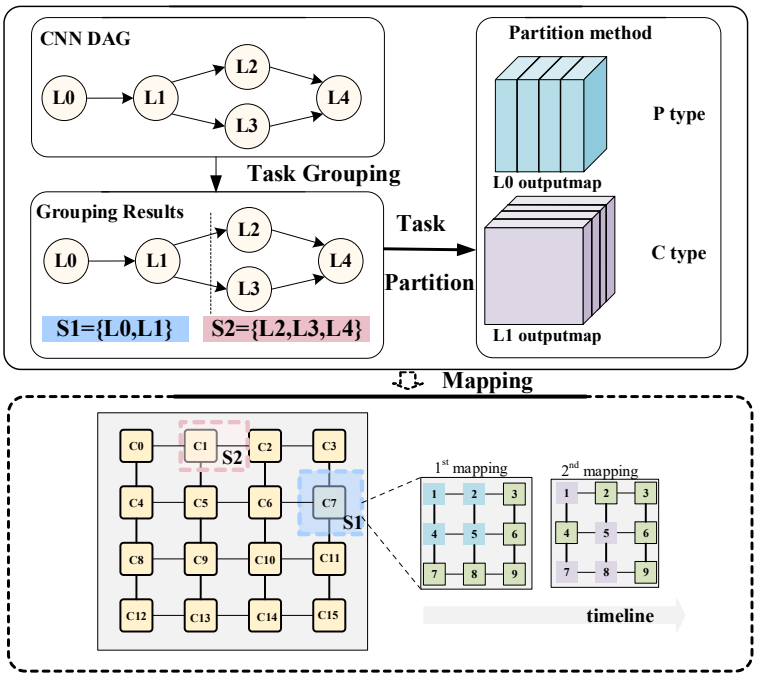


Fig. 2. 基於 Chiplet 的 CNN 加速器任務分配流程。

Var(*G*)代表組間計算負載的變異數，其中 C_i 是 *i- th* 組的計算負載，而 *C* 是所有組的平均計算負載。*Cut*(*G*) 代表組間權重的總和。

本論文提出一種基於 dynamic programming（*DP*）的任務分組方法，如 Algorithm 1 所示。此方法的核心概念是在每個群組內維持 load balancing，同時確保每個任務群組的計算負載不超過預設最大限制 C_{max}，並使群組間通訊資料量最小化。具體步驟如下：該方法首先根據 CNN 應用任務圖（DAG）中每個節點與輸入節點之間的距離進行分群。這會將原本具有非線性相依性的複雜任務圖轉換為具有相對簡單線性關係的結構，因而降低 dynamic programming 演算法的時間複雜度，並簡化後續的任務分組流程。

Algorithm1 DAG Grouping with DP

Input: DAG, C_{max}
Output: $G=\{G_1,G_2,\ldots,G_n\}$

```
linearize DAG
temGroup[]={ }
for i = 1 to k do
    for j=1 to i do
        if C(i,j)≤ Cmax
     tempcost[i]= cost[j]+cost(j,i)
     if tempcost[i]<cost[i]
        cost[i]←tempcost[i]
        tempGroup[i]={tempGroup[j],[vj,…,vi] }
end for
G←tempGroup[n]
Return S
```

接著，使用 DP 將 task graph 中的節點分組，把具有連續 index 的節點分割成多個 subgraph（亦即 task group）。每個 subgraph 的 computational load 必須控制在最大限制 C_{max} 內，以確保在 hardware resource 限制下能有效達到 load balancing。同時，演算法必須最小化跨 group 的 data transmission，以降低 chiplets 之間的 communication volume。

B. Task partitioning

在完成 task grouping 之後，我們會將每個 task group 分配給一個 chiplet。下一步是決定 chiplet 內的 task partitioning，這涉及將 task group 中每個 CNN layer 的 computational tasks 分配給 chiplet 的 PEs。此外，規劃如何 partition output feature map 也至關重要。因此，我們提出一種 task partitioning 方法，該方法首先決定一個 group 所需的 PEs 數量，然後定義 group 內每個 layer 的 partitioning strategy，具體指定每個 layer 的 computation task 的哪一部分分配給每個 PE。

1) Calculation of the required number of PEs

在任務分配期間，目標是讓分配給每個 PE 的計算負載，記為 m_i，盡可能接近但不超過限制值 m。令 *H*、*W* 和 *C* 分別表示某一層輸出 feature map 的高度、寬度和通道數，而 *F* 表示 convolution 或 pooling 的 window size，*S* 則表示 convolution 或 pooling 的 stride。經計算後，該群組的總計算負載記為 C_i。該群組所需的 PE 數量 (N) 可使用以下公式求得，其中 ε 是符合條件的最大整數。

$$N = \varepsilon \tag{5}$$

$$m_i = C_i / \varepsilon \tag{6}$$

Authorized licensed use limited to: National Central University. Downloaded on October 03,2026 at 06:02:59 UTC from IEEE Xplore. Restrictions apply.

$$m_i \le m \tag{7}$$

$$mod\left(H \times W \times C, \varepsilon\right) = 0 \tag{8}$$

$$\varepsilon \in \left[1, H \times W \times C\right] \tag{9}$$

2) Determining the Partitioning Strategy

After determining the number of PEs required for each group, we need to further decide the partitioning strategy for each layer, that is, how to divide the output feature map of each layer into N parts along different dimensions. In this process, a reasonable partitioning strategy is particularly critical, especially the partitioning strategy of adjacent layers directly affects the communication volume between them. Basic partitioning methods can be divided into two main categories: plane partitioning (P) and channel partitioning (C). P-type partitioning is more suitable for large feature layers with shared weights, as it helps parallelize computations along the plane dimension. In contrast, C-type partitioning is more appropriate for wide layers with many weights and fewer activations.

If partitioning is done along a single dimension, there are four possible combinations of adjacent layers. Taking a chiplet with 4 PEs as an example (shown in Fig. 3.), and assuming that the adjacent layers are in the same group, the layer $l-1$ uses 4 PEs to complete its computation, and the layer l continues to use the same 4 PEs for its computation. Suppose the $l-1$ layer uses P-type partitioning and the l layer also uses P-type partitioning. In this case, for PE0, the partial output feature map obtained from the computation of the layer $l-1$ can continue to be used for computing the partial output feature map of the layer l. This means that most of the input data is stored locally, and only a small amount of data needs to be transferred from the neighboring PE1 to PE0 to complete the computation. The data region that PE1 needs to transfer is shown as a yellow bar in the figure, and this region is called the halo area. For other PEs, when computing the output of the layer l, they still need to obtain partial data from the neighboring PE for the $l-1$ layer.

If the layer l uses C-type partitioning, the situation changes. In this case, PE0 needs to obtain the output feature map data of the layer $l-1$ from other PEs. Regardless of whether the layer $l-1$ uses P-type or C-type partitioning, PE0 will need to transfer any data not stored locally from other PEs. Similarly, for other PEs, they will also need to transfer all output feature map data from neighboring PEs, except for the local data. Finally, if the layer $l-1$ uses C-type partitioning and the layer l uses P-type partitioning, in addition to the regular output feature map transfer, extra data, including the halo area, needs to be transmitted to ensure that the computation of the layer l can proceed smoothly.

As shown in Table 1, we present the communication volume calculation methods for four different combinations within the same group. Specifically, if the layer $l-1$ uses P-type partitioning and the l layer also uses P-type partitioning, the size of a single halo region is represented as $(F-S)\times H\times C$. Since adjacent sections after partitioning will both generate halo regions, the total amount of data that needs to be transmitted for the halo regions is $(F-S)\times H\times C\times(N-1)$. If the layer l uses C-type partitioning, the transmission amount for a single activation data is $(N-1)\times H\times W\times C$, and the total data transmission amount is $N-1\times H\times W\times C\times N$. Lastly, if the layer $l-1$ uses C-type partitioning and the layer l uses P-type partitioning, the total data transmission amount is the sum of the two cases above.

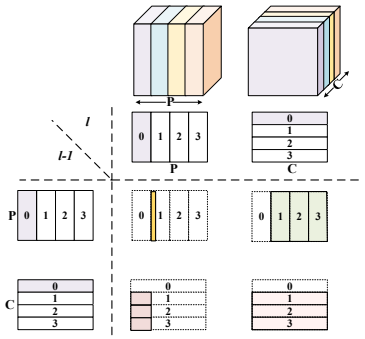


Fig. 3. Analysis of inter-PE communication during layer switching.

If $W/F<N$, partitioning solely in the plane dimension can not divide the output feature map into N parts, so partitioning in the channel dimension (PC) is also needed. To represent the specific partitioning method for PC, we introduce $N1$ to represent the number of partitioning in the plane dimension (P), $N2$ to represent the number of partitioning in the channel dimension (C), and $N=N1\times N2$. Specifically, when $N1=1$, i.e., $N=N_2$, partitioning is only done in the channel dimension. Since the PC partitioning method has five other combinations with the existing P and C partitioning methods, the calculation method for communication volume is similar to those described above.

If the layer $l-1$ and l belong to different groups, and we only consider partitioning in a single dimension, let N represent the total number of the smallest units in the feature map partitioned at the layer $l-1$, and M represent the total number of the smallest units in the feature map partitioned at the layer l. Let M_1 represent the number of partitions in the plane dimension (P) and M_2 represent the number of partitions in the channel dimension (C), so $M=M1\times M2$. Since the layer $l-1$ and l occupy different PEs on different chiplets, there is no data locality, and the communication volume between adjacent layers is as shown in Table 2.

TABLE I. COMMUNICATION MODEL BETWEEN LAYER L-1 AND LAYER L IN SAME GROUP

Partitioning type		**Layer *l***		
		P	***PC***	***C***
Layer *l*-1	P	$(\mathrm{F}-\mathrm{S})\times\mathrm{H}\times\mathrm{C}\times(\mathrm{N}-1)$	$\mathrm{H}\times\mathrm{W}\times\mathrm{C}+(\mathrm{F}-\mathrm{S})\times\mathrm{H}\times\mathrm{C}\times(\mathrm{N1}-1)$	$(\mathrm{N}-1)\times\mathrm{H}\times\mathrm{W}\times\mathrm{C}$
	PC	$\mathrm{H}\times\mathrm{W}\times\mathrm{C}$	$\mathrm{H}\times\mathrm{W}\times\mathrm{C}+$（F-S）$\times\mathrm{H}\times\mathrm{C}\times(\mathrm{N1}-1)$	$(\mathrm{N}-1)\times\mathrm{H}\times\mathrm{W}\times\mathrm{C}$
	C	$(N-1)\times H\times W\times C+(F-S)\times H\times C\times(\mathrm{N}-1)$	$\frac{\mathrm{N}-1}{\mathrm{N1}}\times\mathrm{H}\times\mathrm{W}\times\mathrm{C}+\frac{\mathrm{N}-1}{\mathrm{N}}\times(\mathrm{F}-\mathrm{S})\times\mathrm{H}\times\mathrm{C}(\mathrm{N1}-1)$	$(N-1)\times H\times W\times C$

Authorized licensed use limited to: National Central University. Downloaded on October 03,2026 at 06:02:59 UTC from IEEE Xplore. Restrictions apply.

$$m_i \le m \tag{7}$$

$$mod\left(H \times W \times C, \varepsilon\right) = 0 \tag{8}$$

$$\varepsilon \in \left[1, H \times W \times C\right] \tag{9}$$

2) Determining the Partitioning Strategy

在確定每個群組所需的 PEs 數量之後，我們需要進一步決定每一層的 partitioning strategy，也就是如何沿著不同維度將每一層的 output feature map 劃分為 N 個部分。在此過程中，合理的 partitioning strategy 尤其關鍵，特別是相鄰層的 partitioning strategy 會直接影響它們之間的 communication volume。基本的 partitioning 方法可分為兩大類：plane partitioning (P) 與 channel partitioning (C)。P-type partitioning 更適合具有 shared weights 的大型 feature layers，因為它有助於沿著 plane 維度將 computations 平行化。相對地，C-type partitioning 更適合具有許多 weights 且較少 activations 的 wide layers。

如果 partitioning 沿著單一維度進行，則相鄰層有四種可能的組合。以一個具有 4 個 PEs 的 chiplet 為例（如 Fig. 3. 所示），並假設相鄰層位於同一 group，層 $l-1$ 使用 4 個 PEs 完成其計算，而層 l 繼續使用相同的 4 個 PEs 進行其計算。假設 $l-1$ 層使用 P-type partitioning，且 l 層也使用 P-type partitioning。在這種情況下，對於 PE0，從層 $l-1$ 的計算所獲得的部分輸出 feature map 可以繼續用於計算層 l 的部分輸出 feature map。這意味著大多數輸入資料儲存在本地，只有少量資料需要從鄰近的 PE1 傳輸到 PE0 以完成計算。PE1 需要傳輸的資料區域在圖中以黃色條顯示，此區域稱為 halo area。對於其他 PEs，在計算 層 l 的輸出時，它們仍需要從鄰近的 PE 取得 $l-1$ 層的部分資料。

如果 layer l 使用 C-type partitioning，情況就會改變。在這種情況下，PE0 需要從其他 PEs 取得 layer $l-1$ 的 output feature map data。無論 layer $l-1$ 使用 P-type 或 C-type partitioning 時，PE0 將需要從其他 PE 傳輸任何未儲存在本地的資料。同樣地，對於其他 PE 而言，除了本地資料之外，它們也需要從鄰近的 PE 傳輸所有 output feature map 資料。最後，如果層 $l-1$ 使用 C-type partitioning，而層 l 使用 P-type partitioning，則除了常規的 output feature map 傳輸之外，還需要傳輸額外資料，包括 halo area，以確保層 l 的計算能夠順利進行。

如表 1 所示，我們提出同一 group 內四種不同組合的 communication volume 計算方法。具體而言，如果 layer $l-1$ 使用 P-type partitioning，且 l layer 也使用 P-type partitioning，則單一 halo region 的大小表示為 $(F-S)\times H\times C$。由於 partitioning 後相鄰的 sections 都會產生 halo regions，因此 halo regions 所需傳輸的資料總量為 $(F-S)\times H\times C\times(N-1)$。如果 layer l 使用 C-type partitioning，則單一 activation data 的傳輸量為 $(N-1)\times H\times W\times C$，總資料傳輸量為 $N-1\times H\times W\times C\times N$。最後，如果 layer $l-1$ 使用 C-type partitioning，且 layer l 使用 P-type partitioning，則總資料傳輸量為上述兩種情況之和。

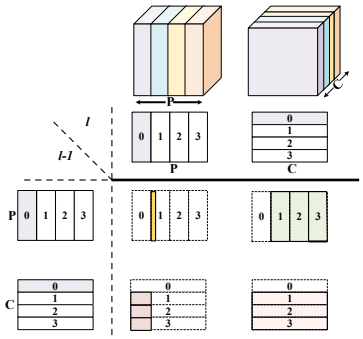


圖 3. 層切換期間 PE 間通訊之分析。

若 $W/F<N$，僅在 plane dimension 進行 partitioning 無法將 output feature map 分成 N 個部分，因此也需要在 channel dimension 進行 partitioning（PC）。為了表示 PC 的具體 partitioning 方法，我們引入 $N1$ 來表示 plane dimension（P）中的 partitioning 數量，$N2$ 來表示 channel dimension（C）中的 partitioning 數量，以及 $N=N1\times N2$。具體而言，當 $N1=1$，即 $N=N_2$ 時，僅在 channel dimension 進行 partitioning。由於 PC partitioning 方法與現有的 P 和 C partitioning 方法還有五種其他組合，communication volume 的計算方法與上述類似。

如果 layer $l-1$ 和 l 屬於不同群組，且我們僅考慮在單一維度中進行 partitioning，令 N 代表在 layer $l-1$ 分割的 feature map 中最小單元的總數，且 M 代表在 layer l 分割的 feature map 中最小單元的總數。令 M_1 代表 plane dimension (P) 中的 partition 數量，且 M_2 代表 channel dimension (C) 中的 partition 數量，因此 M=M1×M2。由於 layer $l-1$ 和 l 在不同 chiplets 上佔用不同的 PEs，因此不存在 data locality，且相鄰 layer 之間的 communication volume 如表 2 所示。

表 I. 同一 GROUP 中 LAYER L-1 與 LAYER L 之間的通訊模型

Partitioning type		**Layer *l***		
		P	***PC***	***C***
Layer *l*-1	P	$(\mathrm{F}-\mathrm{S})\times\mathrm{H}\times\mathrm{C}\times(\mathrm{N}-1)$	$\mathrm{H}\times\mathrm{W}\times\mathrm{C}+(\mathrm{F}-\mathrm{S})\times\mathrm{H}\times\mathrm{C}\times(\mathrm{N1}-1)$	$(\mathrm{N}-1)\times\mathrm{H}\times\mathrm{W}\times\mathrm{C}$
	PC	$\mathrm{H}\times\mathrm{W}\times\mathrm{C}$	$\mathrm{H}\times\mathrm{W}\times\mathrm{C}+$（F-S）$\times\mathrm{H}\times\mathrm{C}\times(\mathrm{N1}-1)$	$(\mathrm{N}-1)\times\mathrm{H}\times\mathrm{W}\times\mathrm{C}$
	C	$(N-1)\times H\times W\times C+(F-S)\times H\times C\times(\mathrm{N}-1)$	$\frac{\mathrm{N}-1}{\mathrm{N1}}\times\mathrm{H}\times\mathrm{W}\times\mathrm{C}+\frac{\mathrm{N}-1}{\mathrm{N}}\times(\mathrm{F}-\mathrm{S})\times\mathrm{H}\times\mathrm{C}(\mathrm{N1}-1)$	$(N-1)\times H\times W\times C$

Authorized licensed use limited to: National Central University. Downloaded on October 03,2026 at 06:02:59 UTC from IEEE Xplore. Restrictions apply.

TABLE II. COMMUNICATION MODEL BETWEEN LAYER l-1 AND LAYER l IN DIFFERENT GROUP

Partitioning type		**Layer *l***		
		P	***PC***	***C***
Layer *l*-1	P	$(F-S)\times H\times C\times M+$ H×W×C	$(M2-1)\times H\times W\times C+$ $(F-S)\times H\times C\times(M1-1)$	$M\times H\times W\times C$
	PC	$H\times W\times C+$ $(F-S)\times H\times C\times M$	M2 ×H × W×C + （F-S）×H×C ×(M1-1)	M ×H × W×C
	C	M×H×W×C + （F-S）×H×C×(M-1)	M2 ×H × W×C + （F-S）×H×C ×(M1-1)	M×H×W×C

IV. EXPERIMENTAL RESULTS

We will use the LeNet-5 model as an example to verify the proposed method. The architecture of the chiplet accelerator used in the experiment consists of a 4×4 array of computing chiplets, with each chiplet containing 4×4 PEs. The maximum computational load per PE (m) is set to 11,000, meaning that during the task grouping phase, the maximum computational load for each chiplet (group) is 11,000×16.

Table 3 shows the results of our proposed task allocation method on the LeNet-5. LeNet-5 is divided into three groups, each assigned to one chiplet. Taking Group 2 (G2) as an example, the task partitioning for each layer is determined. The total computation of G2 is 153,600, requiring at least 14 PEs. However, since the output feature map of Convolutional Layer 2 is 10×10×16, and the output feature map of the pooling layer is 5×5×16, N should be divisible by both of these dimensions, while the computational load per PE should be as close as possible to the value of m. Therefore, G2 is assigned to 16 PEs. For Convolutional Layer 2, the ratio of Width/Feature map size (W/F) is 10/3 < 16. Based on the minimum communication volume between adjacent layers, the values of $N1$ and $N2$ for Convolutional Layer 2 are chosen to be 2 and 8, respectively. For Pooling Layer 2, $N1 = 1$ and $N2 = 16$.

TABLE III. RESULTS OF ALLOCATION METHOD ON LENET-5

	O_i	Group	N	$N1$	$N2$
Convolution Layer1	122304	G1	12	4	3
pooling Layer1	5880			2	6
Convolution Layer2	151600	G2	16	2	8
pooling Layer2	2000			1	16
Convolution Layer3	48120	G3	6	1	6
Fully connected Layer1	10164			1	6
Fully connected Layer2	850			1	6

To facilitate comparison, we explain the uniform allocation method. The architecture parameters of the chiplet-based CNN accelerator are the same as those mentioned above. The number of PEs required for each layer is determined by dividing the computational workload of the layer by the maximum computation capacity of each PE and rounding up. The number of PEs for each layer is assigned layer by layer. To improve utilization, 16 PEs will be included in one chiplet. For example, chiplet 1 contains 12 PEs for convolutional layer 1, 1 PE for the first pooling layer, and 3 PEs for part of convolutional layer 2. The remaining 13 PEs of convolutional layer 2 will be allocated to chiplet2.

Fig. 4 shows the communication volume during the layer switching of LeNet-5 using the proposed method and the uniform allocation method. The task allocation method proposed significantly reduces the communication volume between chiplet. Compared to the uniform allocation method, the proposed method reduces the inter-chiplet communication volume by 59%. Although the intra-chiplet communication volume increases by 30% relative to the uniform allocation method, literature has shown that the inter-chiplet communication delay is approximately 21.3 times the intra-chiplet delay[10]. Therefore, the communication overhead between chiplet plays a more critical role in the performance of chiplet-based accelerators. Fig. 5 shows the normalized communication volume of LeNet-5， AlexNet， VGG16， ResNet18 and ResNet34 using the proposed method and the uniform allocation method. Thus, the proposed task distribution method reduces the total communication volume between and within chiplet, particularly the inter-chiplet communication volume.

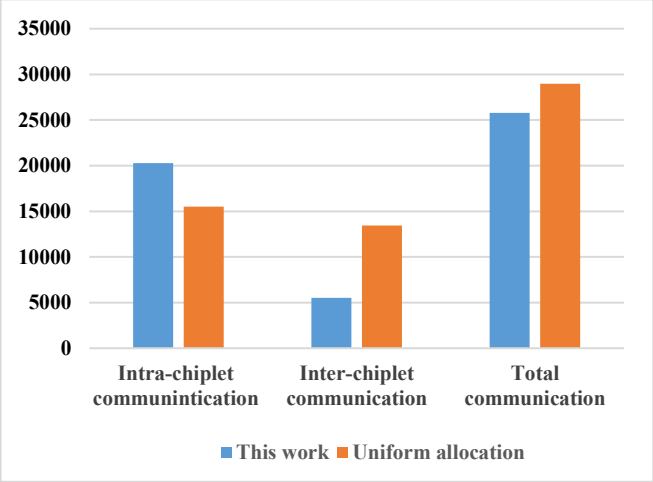


Fig. 4. Communication volume of two task allocation methods on LeNet-5.

Authorized licensed use limited to: National Central University. Downloaded on October 03,2026 at 06:02:59 UTC from IEEE Xplore. Restrictions apply.

表 II. 不同 GROUP 中 LAYER l-1 與 LAYER l 之間的 COMMUNICATION MODEL

Partitioning type		**Layer *l***		
		P	***PC***	***C***
Layer *l*-1	P	$(F-S)\times H\times C\times M+$ H×W×C	$(M2-1)\times H\times W\times C+$ $(F-S)\times H\times C\times(M1-1)$	M × H × W × C
	PC	H × W × C+ (F- S) × H×C×M	M2 ×H × W×C + （F-S）×H×C ×(M1-1)	M ×H × W×C
	C	M×H×W×C + （F-S）×H×C×(M-1)	M2 ×H × W×C + （F-S）×H×C ×(M1-1)	M×H×W×C

IV. 實驗結果

我們將使用 LeNet-5 model 作為範例來驗證所提出的方法。實驗中所使用的 chiplet accelerator 架構由一個 4× 4 的 computing chiplets 陣列組成，每個 chiplet 包含 4× 4 個 PEs。每個 PE 的最大 computational load（m）設定為 11,000，這表示在 task grouping phase 期間，每個 chiplet（group）的最大 computational load 為 11,000× 16。

表 3 顯示我們提出的 task allocation 方法在 LeNet-5 上的結果。LeNet-5 被分為三個 groups，每個分配給一個 chiplet。以 Group 2 (G2) 為例，每一層的 task partitioning 被決定。G2 的總 computation 為 153,600，至少需要 14 個 PEs。然而，由於 Convolutional Layer 2 的 output feature map 為 10× 10× 16，而 pooling layer 的 output feature map 為 5× 5× 16，N 應同時可被這兩個維度整除，而每個 PE 的 computational load 應盡可能接近 m 的值。因此，G2 被分配給 16 個 PEs。對於 Convolutional Layer 2，Width/Feature map size (W/F) 的比例為 10/3 < 16。基於相鄰層之間的最小 communication volume，Convolutional Layer 2 的 $N1$ 與 $N2$ 之值分別選為 2 與 8。對於 Pooling Layer 2，$N1 = 1$ 與 $N2 = 16$。

表 III. ALLOCATION METHOD 在 LENET-5 上的結果

	O_i	Group	N	$N1$	$N2$
Convolution Layer1	122304	G1	12	4	3
pooling Layer1	5880			2	6
Convolution Layer2	151600	G2	16	2	8
pooling Layer2	2000			1	16
Convolution Layer3	48120	G3	6	1	6
Fully connected Layer1	10164			1	6
Fully connected Layer2	850			1	6

為了便於比較，我們說明 uniform allocation 方法。以 chiplet 為基礎的 CNN accelerator 的架構參數與上述相同。每一層所需的 PEs 數量，是將該層的計算工作量除以每個 PE 的最大計算能力並向上取整來決定。數量每層的 PEs 會逐層分配。為了提升利用率，一個 chiplet 將包含 16 個 PEs。例如，chiplet 1 包含 convolutional layer 1 的 12 個 PEs、第一個 pooling layer 的 1 個 PE，以及 convolutional layer 2 一部分的 3 個 PEs。convolutional layer 2 其餘的 13 個 PEs 將分配給 chiplet2。

圖 4 顯示使用所提出方法與 uniform allocation method 時，LeNet-5 在 layer switching 期間的 communication volume。所提出的 task allocation method 顯著降低了 chiplet 之間的 communication volume。與 uniform allocation method 相比，所提出方法將 inter-chiplet communication volume 降低了 59%。儘管相對於 uniform allocation method，intra-chiplet communication volume 增加了 30%，文獻顯示 inter-chiplet communication delay 約為 intra-chiplet delay 的 21.3 倍[10]。因此，chiplet 之間的 communication overhead 在 chiplet-based accelerators 的效能中扮演更關鍵的角色。圖 5 顯示使用所提出方法與 uniform allocation method 時，LeNet-5、AlexNet、VGG16、ResNet18 與 ResNet34 的 normalized communication volume。因此，所提出的 task distribution method 降低了 chiplet 之間與內部的總 communication volume，尤其是 inter-chiplet communication volume。

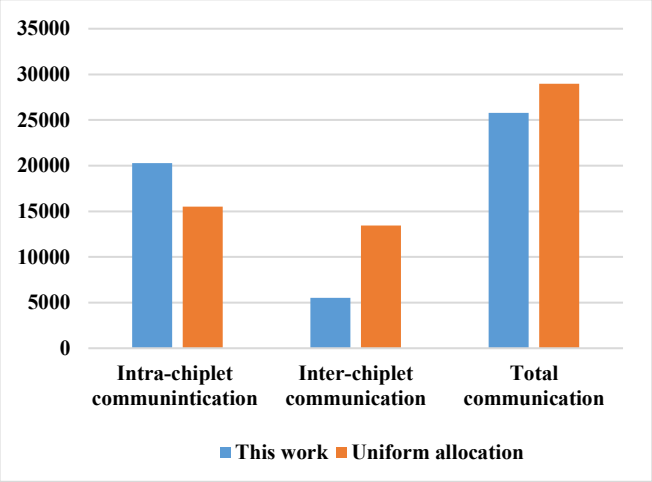


圖 4. 兩種 task allocation 方法在 LeNet-5 上的 communication volume。

Authorized licensed use limited to: National Central University. Downloaded on October 03,2026 at 06:02:59 UTC from IEEE Xplore. Restrictions apply.

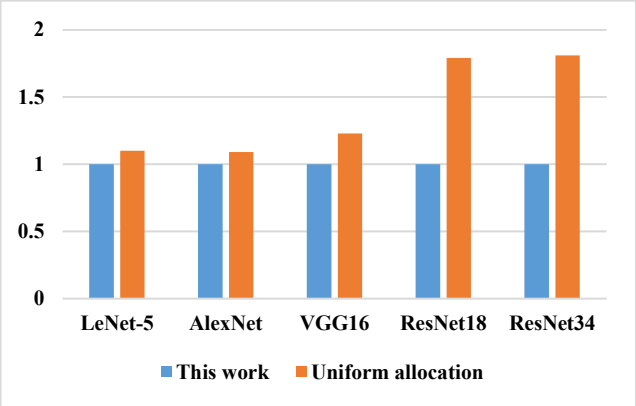


Fig. 5. Normalized communication volume of different CNN models.

V. Conclusion

This paper proposes a task allocation method for chiplet-based CNN accelerators. The method first groups the network layers in a reasonable way and assigns each group to a chiplet, effectively reducing the communication overhead between chiplets. Based on the computational resource constraints of each chiplet, the number of PEs required for each group is determined and the layer partitioning strategy is optimized. Experimental results show that, compared with the uniform task allocation method, this approach significantly reduces the communication overhead between chiplets and optimizes the overall communication overhead both within and between chiplets. System latency and power consumption exhibit a significant positive correlation with communication overhead, optimizing the communication can effectively improve two critical performance metrics.

References

[1] Gu, Jiuxiang, "Recent advances in convolutional neural networks," Pattern Recognition, vol. 77, pp. 354-377, May. 2018.

[2] C. Pham-Quoc, T. Pham-Dinh, B. Kieu-Do-Nguyen, "Efficient Random Forest Acceleration for Edge Computing Platforms with FPGA Technology," Journal of Advances in Information Technology, vol. 15, no. 2, pp. 195-201, Feb. 2024.

[3] Y. Han, H. Xu, M. Lu, et al, "The big chip: Challenge, model and architecture," Fundamental Research, vol. 4, no. 6, pp. 1431-1441, Nov. 2024.

[4] Y. Feng and K. Ma, "Chiplet actuary: a quantitative cost model and multi-chiplet architecture exploration," in Proceedings of the 59th ACM/IEEE Design Automation Conference, San Francisco California: ACM, pp. 121-126, Jul. 2022.

[5] G. Krishnan, S. K. Mandal, M. Pannala, et al, "SIAM: Chiplet-based scalable in-memory acceleration with mesh for deep neural networks," ACM transactions on embedded computing systems (TECS), vol. 20, no. 5s, pp. 1-24, Oct. 2021.

[6] Z. Tan, H. Cai, R. Dong, and K. Ma, "NN-baton: DNN workload orchestration and chiplet granularity exploration for multi-chip accelerators," in 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA), pp. 1013-1026, Jun. 2021.

[7] J. Zhang, X. Fan, Y. Ye, et al., "INDM: Chiplet-based interconnect network and dataflow mapping for DNN accelerators," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 43, no. 4, pp. 1107-1120, Apr. 2024.

[8] Y. S. Shao, J. Clemons, R. Venkatesan, et al, "Simba: Scaling deep learning inference with multi-chip-module-based architecture," in Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, pp. 14-27, Oct. 2019.

[9] J. Cai, Z. Wu, S. Peng, et al, "Gemini: Mapping and architecture co-exploration for large-scale DNN chiplet accelerators," in 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA), pp. 156-171, Mar. 2024.

[10] M. A. Kabir, D. Petranovic, and Y. Peng, "Coupling Extraction and Optimization for Heterogeneous 2.5D Chiplet-Package Co-Design," in 2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD), pp. 1-8, Nov. 2020.

Authorized licensed use limited to: National Central University. Downloaded on October 03,2026 at 06:02:59 UTC from IEEE Xplore. Restrictions apply.

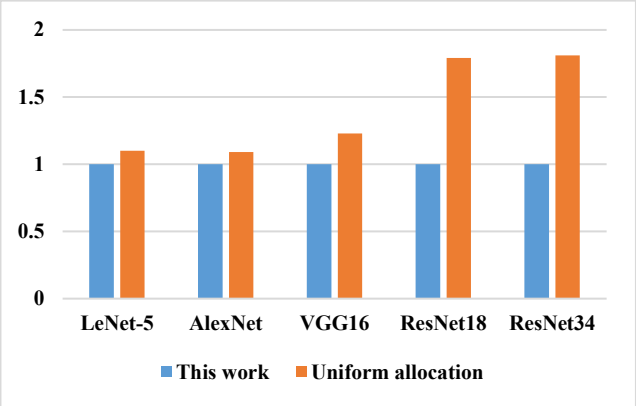


圖 5. 不同 CNN 模型的正規化通訊量。

V. 結論

本論文提出一種針對基於 chiplet 的 CNN 加速器之任務分配方法。該方法首先以合理方式將網路層分組，並將每個群組分配給一個 chiplet，有效降低 chiplet 之間的 communication overhead。根據每個 chiplet 的 computational resource constraints，決定每個群組所需的 PEs 數量，並最佳化 layer partitioning strategy。實驗結果顯示，與 uniform task allocation method 相比，此方法顯著降低 chiplet 之間的 communication overhead，並最佳化 chiplet 內部與 chiplet 之間的整體 communication overhead。System latency 與 power consumption 與 communication overhead 呈現顯著正相關，最佳化 communication 可有效改善兩項關鍵效能指標。

參考文獻

[1] Gu, Jiuxiang, “卷積神經網路（convolutional neural networks）的最新進展，” Pattern Recognition，第 77 卷，第 354-377 頁，2018 年 5 月。[2] C. Pham-Quoc、T. Pham-Dinh、B. Kieu-Do-Nguyen，“具 FPGA 技術之 Edge Computing 平台的高效 Random Forest 加速，” Journal of Advances in Information Technology，第 15 卷，第 2 期，第 195-201 頁，2024 年 2 月。[3] Y. Han、H. Xu、M. Lu 等，“大晶片：挑戰、模型與架構，” Fundamental Research，第 4 卷，第 6 期，第 1431-1441 頁，2024 年 11 月。[4] Y. Feng 與 K. Ma，“Chiplet actuary：定量成本模型與 multi-chiplet 架構探索，” 收錄於 Proceedings of the 59th ACM/IEEE Design Automation Conference，San Francisco California：ACM，第 121-126 頁，2022 年 7 月。[5] G. Krishnan、S. K. Mandal、M. Pannala 等，“SIAM：用於 deep neural networks 的 Chiplet-based scalable in-memory acceleration with mesh，” ACM transactions on embedded computing systems (TECS)，第 20 卷，第 5s 期，第 1-24 頁，2021 年 10 月。[6] Z. Tan、H. Cai、R. Dong 與 K. Ma，“NN-baton：針對 multi-chip accelerators 的 DNN workload orchestration 與 chiplet granularity exploration，” 收錄於 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)，第 1013-1026 頁，2021 年 6 月。[7] J. Zhang、X. Fan、Y. Ye 等，“INDM：用於 DNN accelerators 的 Chiplet-based interconnect network 與 dataflow mapping，” IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems，第 43 卷，第 4 期，第 1107-1120 頁，2024 年 4 月。[8] Y. S. Shao、J. Clemons、R. Venkatesan 等，“Simba：以 multi-chip-module-based architecture 擴展 deep learning inference，” 收錄於 Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture，第 14-27 頁，2019 年 10 月。[9] J. Cai、Z. Wu、S. Peng 等，“Gemini：針對大規模 DNN chiplet accelerators 的 mapping 與 architecture co-exploration，” 收錄於 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)，第 156-171 頁，2024 年 3 月。[10] M. A. Kabir、D. Petranovic 與 Y. Peng，“用於異質 2.5D Chiplet-Package Co-Design 的 Coupling Extraction 與 Optimization，” 收錄於 2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)，第 1-8 頁，2020 年 11 月。

Authorized licensed use limited to: National Central University. Downloaded on October 03,2026 at 06:02:59 UTC from IEEE Xplore. Restrictions apply.